CampusPrint

University print desk simulator
Software Design Patterns · Final project defense
Amir Baiseitov · SE-2505 · November 2026

1. Introduction

A university print desk takes orders for individual pages or whole documents. A student may also ask for several copies, a cover or binding. The operator needs to know what to charge, which order to print next and what work has actually finished.

I developed CampusPrint as an individual Java project. My aim was to use six design patterns to answer those questions in one console application. The program simulates a single printer. It calculates prices and page counts, maintains a queue in memory and records completed jobs. It can also simulate a paper jam.

2. Main body

2.1 The reference orders

The operator first accepts a thesis and then an application. Page ranges include both endpoints. Printing costs 20 KZT per printed page, a cover costs 100 KZT per copy, and binding costs 200 KZT per copy.

Order	Pages selected	Copies	Printed pages	Services	Total
1. Thesis	1-80 of 80	1	80	Printing, cover, binding	1,900 KZT
2. Application	2-3 of 5	2	4	Printing	80 KZT

The application is a useful case for checking the design. Selecting pages 2 and 3 in two copies produces four printed pages. Billing uses that number to calculate 80 KZT. The shortest-job policy uses it to select the application. Once the printer finishes, the statistics subscriber adds exactly four pages. All three operations obtain the volume from the same `PrintJob`.

The thesis illustrates a different connection. Adding binding increases the price on the customer's receipt by 200 KZT and adds "Binding" to the operator's job ticket. Both documents use the services attached to the accepted order.

2.2 Pattern overview

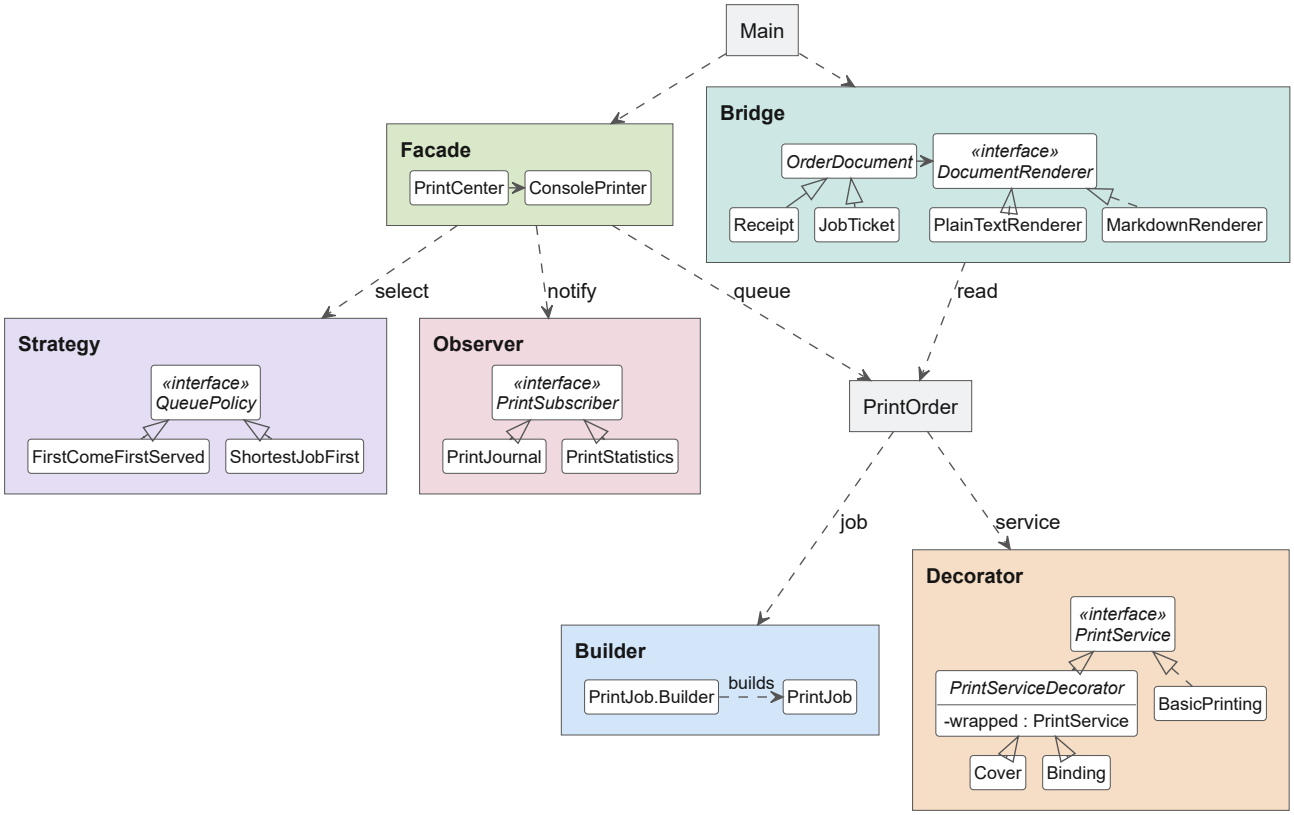


Figure 1. Colors identify the same pattern groups throughout. PrintCenter acts as the Facade, holds the selected policy as the Strategy context and publishes completions as the Observer subject. PrintOrder links a job to its services.

Builder. The thesis uses the defaults of one copy and the whole document. The application sets both options. When `build()` runs, it checks the final range and copy count together. It rejects pages outside the document, a nonpositive quantity or a total volume exceeding the per-job page limit. Pricing and scheduling then read the same immutable job.

```
return new PrintJob.Builder("Application", 5).range(2, 3).copies(copies).build();
```

Decorator. `BasicPrinting` creates the charge for the printed pages. `Cover` and `Binding` each wrap another `PrintService`, retain its line items and add one finishing charge per copy. For the thesis the calculation is 80 × 20 + 100 + 200 = 1,900 KZT. A second cover is rejected even if another decorator sits between the two covers.

```
return center.acceptOrder(thesis, new Binding(new Cover(new BasicPrinting())));
```

2.3 Implementation class diagram

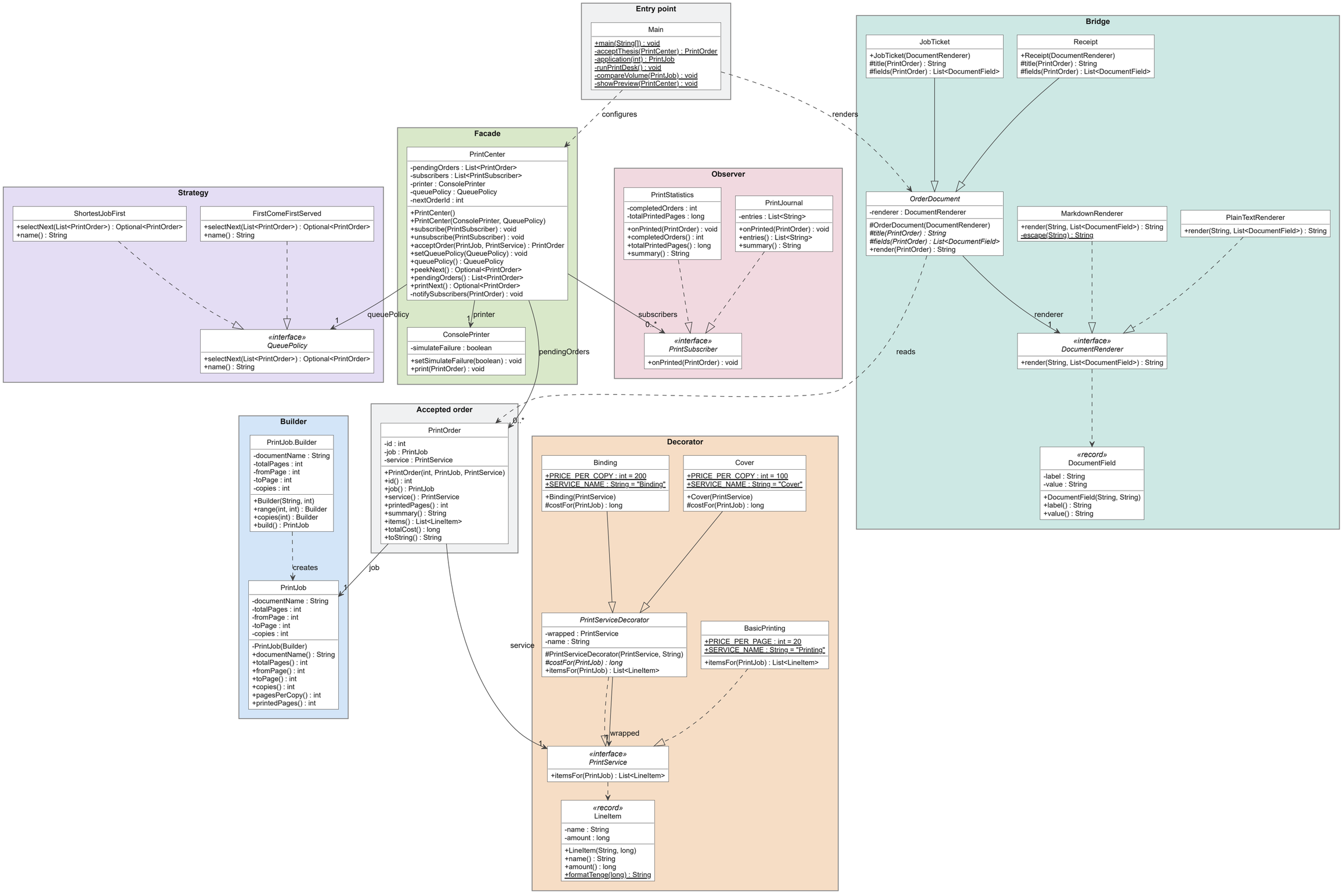


Figure 2. The 24 application source files define 25 types, including the nested PrintJob.Builder. The diagram shows their fields, constructors and operations.

Visibility is marked with + (public), - (private) and # (protected). Underlining marks a static member. Parameter types appear inside parentheses. Hollow triangles point to a superclass or implemented interface. Solid associations show stored references, including the pending orders and subscribers held by PrintCenter. Method signatures also show types used as arguments and results.

2.4 Selecting and completing a job

Strategy. `PrintCenter.peekNext()` asks the active `QueuePolicy` to select from the pending orders in arrival order. FIFO takes the first entry. Shortest job first compares printed page counts, including copies. It replaces its candidate only for a strictly smaller job, so the earlier order wins a tie. Finishing time is outside this estimate. Switching policies leaves both jobs pending.

Facade. The operator calls `PrintCenter.printNext()`. The center uses the same preview to select the order, sends it to the printer and removes it only after printing succeeds. If the printer throws a paper-jam exception, the order remains pending for a retry.

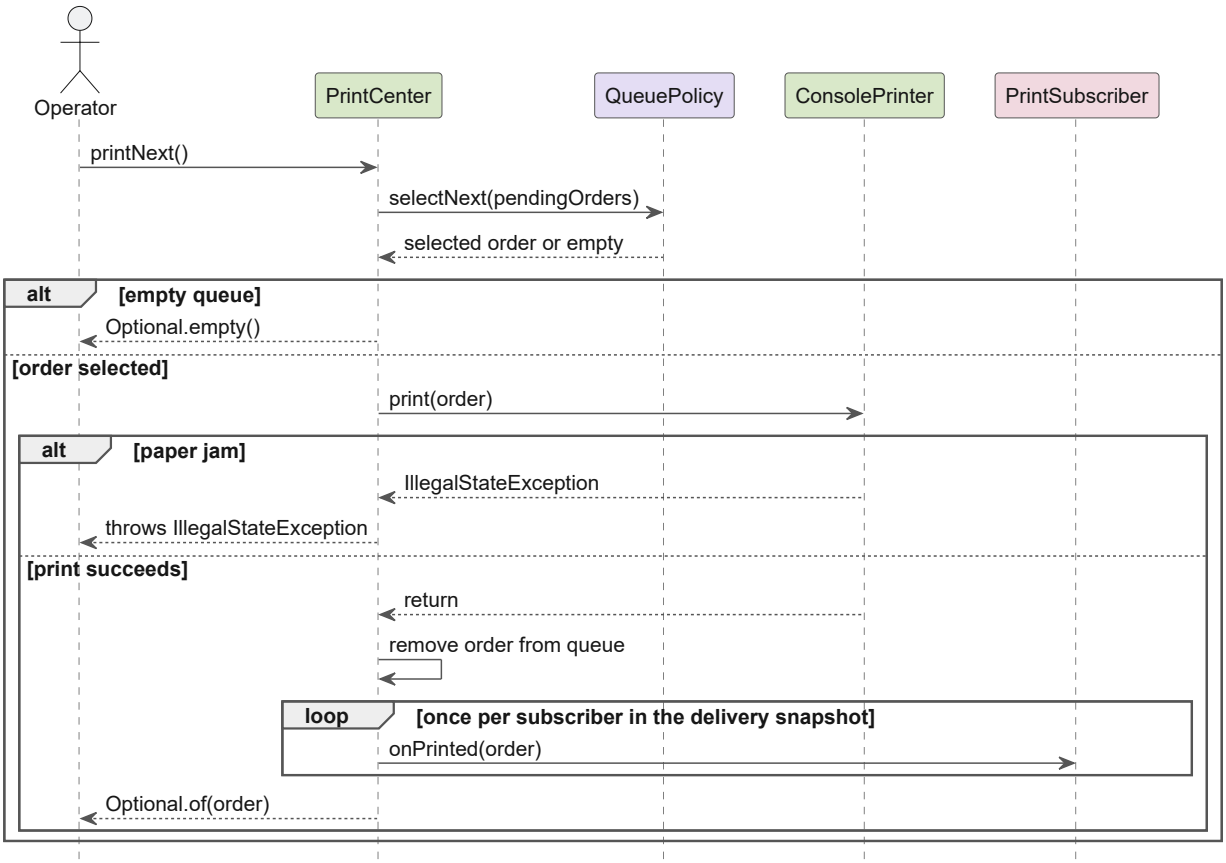


Figure 3. The three outcomes of printNext(). A paper jam leaves the queue intact. A successful print triggers one callback per registered subscriber.

Observer. `PrintJournal` records the completed order's number and title. `PrintStatistics` increments the job and page totals. Both receive the order through `onPrinted(order)`. The center takes a snapshot of the subscriber list so an observer can unsubscribe during its callback without causing another observer to miss that print.

The retry check uses two ten-page orders, Handout and Worksheet. With a simulated jam on Handout, both remain in the queue and the journal is empty. After the fault is cleared, the journal records Handout followed by Worksheet and the counter reads 20 pages. This checks the point at which the facade publishes completion.

2.5 Receipt and job ticket

Bridge. The customer needs an itemized bill. The operator needs a page range, copy count and finishing instructions. `Receipt` and `JobTicket` choose the relevant fields from the same `PrintOrder`. Their base class, `OrderDocument`, passes the title and fields to its `DocumentRenderer`.

The text renderer aligns labels and values. The Markdown renderer puts the same fields into a table and escapes vertical bars in field values. A receipt pairs each service with its charge. A job ticket lists the same services as operations and gives the operator the range and copy count. Each renderer receives just a title and labeled fields, so it can format either document.

```
CampusPrint / Print desk
#1 Thesis | copies: 1 | pages: 80 | 1,900 KZT
#2 Application | copies: 2 | pages: 4 | 80 KZT

First come, first served -> #1 Thesis (80 pages)
Shortest job first -> #2 Application (4 pages)
Printed #2 Application (4 pages)
Journal: #2 Application
Completed orders: 1 | Printed pages: 4

Receipt #1 / Thesis
  Printing        1,600 KZT
  Cover           100 KZT
  Binding         200 KZT
  Total           1,900 KZT

Job ticket #1 / Thesis
  Document        Thesis
  Page range      1-80 of 80
  Copies          1
  Printed pages   80
  Operations      Printing, Cover, Binding

### Receipt #1 / Thesis

| Field | Value |
| --- | --- |
| Printing | 1,600 KZT |
| Cover | 100 KZT |
| Binding | 200 KZT |
| Total | 1,900 KZT |

### Job ticket #1 / Thesis

| Field | Value |
| --- | --- |
| Document | Thesis |
| Page range | 1-80 of 80 |
| Copies | 1 |
```

Figure 4. The first print adds four pages to the counter. For the pending thesis, the receipt shows 100 KZT for a cover and 200 KZT for binding. The job ticket lists both operations.

2.6 Application run

From the project root, compile and run the demonstration with JDK 17 or later.

```
javac --release 17 -encoding UTF-8 -d out -sourcepath src src/campusprint/Main.java
java -cp out campusprint.Main
```

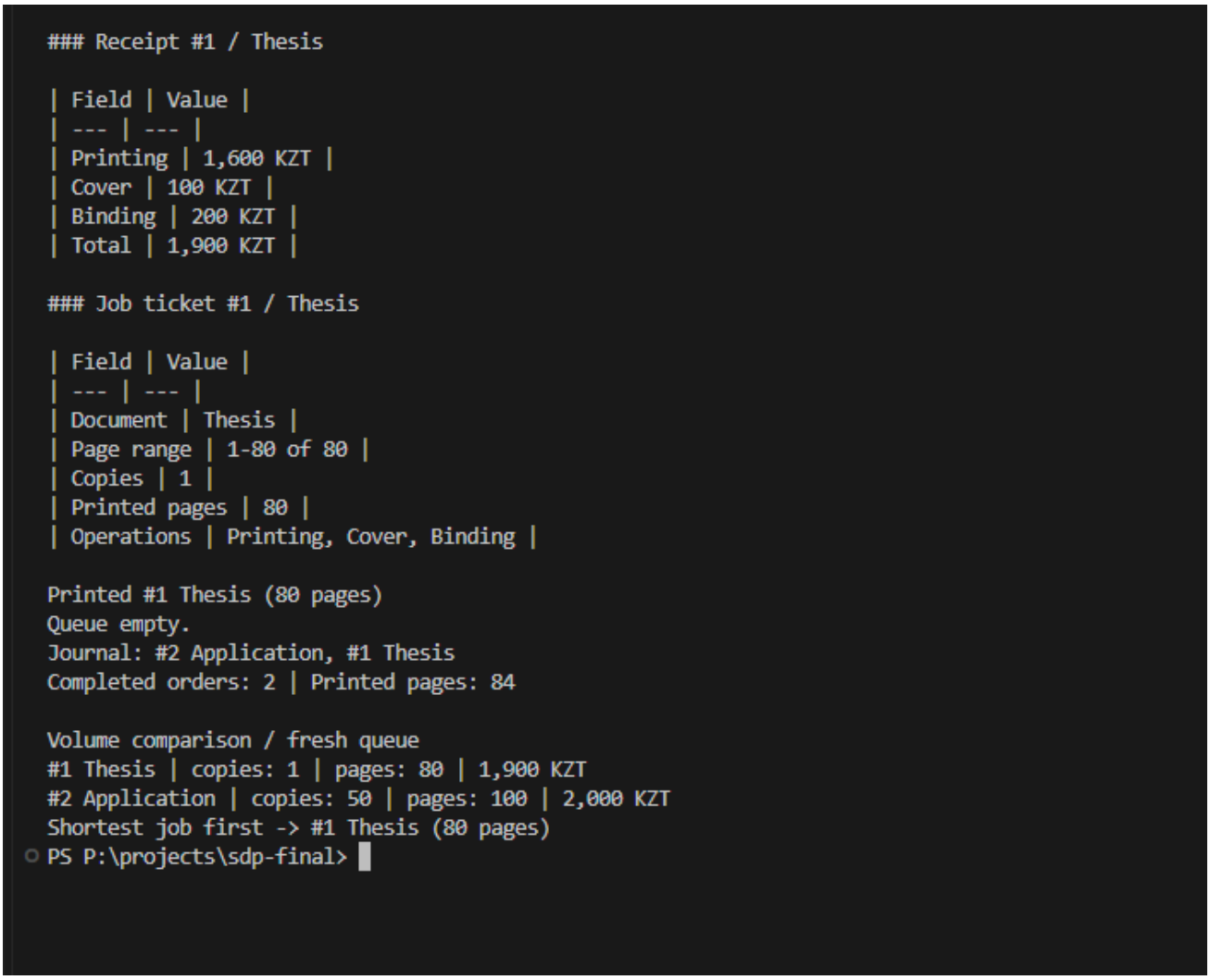


Figure 5. The Markdown ticket retains all five fields from the plain-text ticket. The first queue finishes with two orders and 84 printed pages. In the fresh queue, the final line previews the 80-page thesis ahead of the 100-page application.

To test the selection yourself, run `java -cp out campusprint.Main 39` and then repeat with `40`. This argument changes the copy count only in the fresh queue. At 39 copies, two pages per copy give the application 78 pages and it wins the preview. At 40 copies both jobs have 80 pages, so the earlier thesis wins. Both orders remain pending in this comparison.

2.7 Verification

The regression checks run independently of the demonstration. They throw an assertion error if an observed value differs from the expected value.

```
javac --release 17 -encoding UTF-8 -cp out -d out test/campusprint/PrintCenterChecks.java
java -cp out campusprint.PrintCenterChecks
```

Check	Observed result
Invalid range or copy count	An inverted range, a range beyond the document and zero copies are rejected before acceptance.
Numeric limits	A batch of 65,536 pages × 65,536 copies is rejected. Prices and cumulative page totals above the 32-bit limit retain their value.
Combined services	Three copies of four pages with both finishes cost 1,140 KZT. Reading the quote twice preserves the original line items.
Jam, recovery and empty queue	A failed print leaves the queue, journal and statistics unchanged. After recovery both jobs complete once. An empty-queue request adds no completion.
Subscription change	A subscriber removes itself during a callback. The remaining subscriber still receives the event. A new subscriber starts on the next event.
Document content and format	One test renderer receives two different field lists for the same order. The receipt totals 320 KZT and the ticket specifies six pages and a cover. A title containing a vertical bar and a line break stays in one Markdown table cell.
Live queue selection	At 10 copies the comparison previews the 20-page application and quotes 400 KZT. At the default 50 copies it previews the 80-page thesis.

3. Conclusion

CampusPrint applies all six patterns to the same orders. In the reference run the four-page application prints first under shortest job first, the journal records order 2, and the page counter increases by four. The thesis then prints with the same finishing services shown on its receipt. The failed-print check also confirms that accepting an order and completing it are separate events.

4. Future work

The queue currently lasts for one program run. Persisting accepted and completed orders would allow recovery after a restart. The recovery design would need to distinguish a pending job from a job that reached the printer just before the application stopped. A second extension would add a maximum waiting time to the shortest-job policy so that repeated small jobs cannot postpone a long thesis indefinitely.

5. Java source code

The listing contains all application files under `src/campusprint`. The accompanying ZIP also includes the regression checks in `test/campusprint`.

campusprint/Main.java

```
package campusprint;

import campusprint.center.PrintCenter;
import campusprint.document.JobTicket;
import campusprint.document.MarkdownRenderer;
import campusprint.document.PlainTextRenderer;
import campusprint.document.Receipt;
import campusprint.domain.PrintJob;
import campusprint.domain.PrintOrder;
import campusprint.observer.PrintJournal;
import campusprint.observer.PrintStatistics;
import campusprint.pricing.BasicPrinting;
import campusprint.pricing.Binding;
import campusprint.pricing.Cover;
import campusprint.queue.ShortestJobFirst;

public final class Main {
    public static void main(String[] args) {
        try {
            int comparisonCopies = args.length == 0 ? 50 : Integer.parseInt(args[0]);
            if (args.length > 1 || comparisonCopies < 1) {
                throw new IllegalArgumentException("Expected one positive copy count.");
            }
            PrintJob comparisonJob = application(comparisonCopies);
            runPrintDesk();
            compareVolume(comparisonJob);
        } catch (IllegalArgumentException invalid) {
            System.err.println(invalid.getMessage());
            System.err.println("Usage: java -cp out campusprint.Main [comparisonCopies]");
            System.exit(2);
        }
    }

    private static PrintOrder acceptThesis(PrintCenter center) {
        PrintJob thesis = new PrintJob.Builder("Thesis", 80).build();
        return center.acceptOrder(thesis, new Binding(new Cover(new BasicPrinting())));
    }

    private static PrintJob application(int copies) {
        return new PrintJob.Builder("Application", 5).range(2, 3).copies(copies).build();
    }

    private static void runPrintDesk() {
        PrintCenter center = new PrintCenter();
        PrintJournal journal = new PrintJournal();
        PrintStatistics statistics = new PrintStatistics();
        center.subscribe(journal);
        center.subscribe(statistics);

        System.out.println("CampusPrint / Print desk");
        PrintOrder thesis = acceptThesis(center);
        PrintOrder application = center.acceptOrder(application(2), new BasicPrinting());
        System.out.println(thesis);
        System.out.println(application);
        System.out.println();
        showPreview(center);
        center.setQueuePolicy(new ShortestJobFirst());
        showPreview(center);
```

```
        center.printNext();
        System.out.println(journal.summary());
        System.out.println(statistics.summary());

        PlainTextRenderer text = new PlainTextRenderer();
        MarkdownRenderer markdown = new MarkdownRenderer();
        System.out.println();
        System.out.println(new Receipt(text).render(thesis));
        System.out.println();
        System.out.println(new JobTicket(text).render(thesis));
        System.out.println();
        System.out.println(new Receipt(markdown).render(thesis));
        System.out.println();
        System.out.println(new JobTicket(markdown).render(thesis));
        System.out.println();

        center.printNext();
        center.printNext();
        System.out.println(journal.summary());
        System.out.println(statistics.summary());
    }

    private static void compareVolume(PrintJob application) {
        PrintCenter center = new PrintCenter();
        PrintOrder thesis = acceptThesis(center);
        PrintOrder other = center.acceptOrder(application, new BasicPrinting());
        center.setQueuePolicy(new ShortestJobFirst());
        System.out.println();
        System.out.println("Volume comparison / fresh queue");
        System.out.println(thesis);
        System.out.println(other);
        showPreview(center);
    }

    private static void showPreview(PrintCenter center) {
        PrintOrder next = center.peekNext().orElseThrow();
        System.out.println(center.queuePolicy().name() + " -> " + next.summary());
    }
}
```

campusprint/center/ConsolePrinter.java

```
package campusprint.center;

import campusprint.domain.PrintOrder;

public final class ConsolePrinter {
    private boolean simulateFailure;

    public void setSimulateFailure(boolean simulateFailure) {
        this.simulateFailure = simulateFailure;
    }

    public void print(PrintOrder order) {
        if (simulateFailure) {
            throw new IllegalStateException("Paper jam on order #" + order.id() + ".");
        }
        System.out.println("Printed " + order.summary());
    }
}
```

campusprint/center/PrintCenter.java

```
package campusprint.center;

import campusprint.domain.PrintJob;
```

```
import campusprint.domain.PrintOrder;
import campusprint.observer.PrintSubscriber;
import campusprint.pricing.PrintService;
import campusprint.queue.FirstComeFirstServed;
import campusprint.queue.QueuePolicy;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.Optional;

public final class PrintCenter {
    private final List<PrintOrder> pendingOrders = new ArrayList<>();
    private final List<PrintSubscriber> subscribers = new ArrayList<>();
    private final ConsolePrinter printer;
    private QueuePolicy queuePolicy;
    private int nextOrderId = 1;

    public PrintCenter() {
        this(new ConsolePrinter(), new FirstComeFirstServed());
    }

    public PrintCenter(ConsolePrinter printer, QueuePolicy initialPolicy) {
        this.printer = Objects.requireNonNull(printer, "printer");
        this.queuePolicy = Objects.requireNonNull(initialPolicy, "initialPolicy");
    }

    public void subscribe(PrintSubscriber subscriber) {
        Objects.requireNonNull(subscriber, "subscriber");
        if (!subscribers.contains(subscriber)) {
            subscribers.add(subscriber);
        }
    }

    public void unsubscribe(PrintSubscriber subscriber) {
        subscribers.remove(subscriber);
    }

    public PrintOrder acceptOrder(PrintJob job, PrintService service) {
        PrintOrder order = new PrintOrder(nextOrderId, job, service);
        pendingOrders.add(order);
        nextOrderId++;
        return order;
    }

    public void setQueuePolicy(QueuePolicy policy) {
        queuePolicy = Objects.requireNonNull(policy, "policy");
    }

    public QueuePolicy queuePolicy() {
        return queuePolicy;
    }

    public Optional<PrintOrder> peekNext() {
        return queuePolicy.selectNext(pendingOrders());
    }

    public List<PrintOrder> pendingOrders() {
        return List.copyOf(pendingOrders);
    }

    public Optional<PrintOrder> printNext() {
        Optional<PrintOrder> next = peekNext();
        if (next.isEmpty()) {
            System.out.println("Queue empty.");
            return next;
```

```
        }
        PrintOrder order = next.orElseThrow();
        printer.print(order);
        pendingOrders.remove(order);
        notifySubscribers(order);
        return next;
    }

    private void notifySubscribers(PrintOrder order) {
        // Subscriptions changed during delivery take effect on the next print.
        for (PrintSubscriber subscriber : List.copyOf(subscribers)) {
            try {
                subscriber.onPrinted(order);
            } catch (RuntimeException failure) {
                System.err.printf("Notification failed for order #%d: %s%n", order.id(),
failure.getMessage());
            }
        }
    }
}
```

campusprint/document/DocumentField.java

```
package campusprint.document;

import java.util.Objects;

public record DocumentField(String label, String value) {
    public DocumentField {
        if (label == null || label.isBlank()) {
            throw new IllegalArgumentException("A document field needs a label.");
        }
        Objects.requireNonNull(value, "value");
    }
}
```

campusprint/document/DocumentRenderer.java

```
package campusprint.document;

import java.util.List;

public interface DocumentRenderer {
    String render(String title, List<DocumentField> fields);
}
```

campusprint/document/JobTicket.java

```
package campusprint.document;

import campusprint.domain.PrintJob;
import campusprint.domain.PrintOrder;
import campusprint.pricing.LineItem;
import java.util.List;
import java.util.stream.Collectors;

public final class JobTicket extends OrderDocument {
    public JobTicket(DocumentRenderer renderer) {
        super(renderer);
    }

    @Override
    protected String title(PrintOrder order) {
        return "Job ticket #" + order.id() + " / " + order.job().documentName();
    }

    @Override
```

```
    protected List<DocumentField> fields(PrintOrder order) {
        PrintJob job = order.job();
        String operations =
order.items().stream().map(LineItem::name).collect(Collectors.joining(", "));
        return List.of(
                new DocumentField("Document", job.documentName()),
                new DocumentField("Page range", "%d-%d of %d".formatted(job.fromPage(),
job.toPage(), job.totalPages())),
                new DocumentField("Copies", Integer.toString(job.copies())),
                new DocumentField("Printed pages", Integer.toString(job.printedPages())),
                new DocumentField("Operations", operations));
    }
}
```

campusprint/document/MarkdownRenderer.java

```
package campusprint.document;

import java.util.List;

public final class MarkdownRenderer implements DocumentRenderer {
    @Override
    public String render(String title, List<DocumentField> fields) {
        StringBuilder table = new StringBuilder("### ").append(escape(title));
        table.append("\n\n| Field | Value |\n| --- | --- |");
        for (DocumentField field : fields) {
            table.append("\n| ").append(escape(field.label()))
                    .append(" | ").append(escape(field.value())).append(" |");
        }
        return table.toString();
    }

    private static String escape(String text) {
        return text.replace("\\", "\\\\").replace("|", "\\|").replaceAll("\\R", " ")
                .replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;")
                .replace("*", "\\*").replace("_", "\\_").replace("`", "\\`")
                .replace("[", "\\[").replace("]", "\\]").replace("~", "\\~");
    }
}
```

campusprint/document/OrderDocument.java

```
package campusprint.document;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Objects;

public abstract class OrderDocument {
    private final DocumentRenderer renderer;

    protected OrderDocument(DocumentRenderer renderer) {
        this.renderer = Objects.requireNonNull(renderer, "renderer");
    }

    protected abstract String title(PrintOrder order);

    protected abstract List<DocumentField> fields(PrintOrder order);

    public final String render(PrintOrder order) {
        return renderer.render(title(order), fields(order));
    }
}
```

campusprint/document/PlainTextRenderer.java

```
package campusprint.document;

import java.util.List;

public final class PlainTextRenderer implements DocumentRenderer {
    @Override
    public String render(String title, List<DocumentField> fields) {
        StringBuilder text = new StringBuilder(title);
        for (DocumentField field : fields) {
            text.append(System.lineSeparator())
                    .append("  %-16s %s".formatted(field.label(), field.value()));
        }
        return text.toString();
    }
}
```

campusprint/document/Receipt.java

```
package campusprint.document;

import campusprint.domain.PrintOrder;
import campusprint.pricing.LineItem;
import java.util.ArrayList;
import java.util.List;

public final class Receipt extends OrderDocument {
    public Receipt(DocumentRenderer renderer) {
        super(renderer);
    }

    @Override
    protected String title(PrintOrder order) {
        return "Receipt #" + order.id() + " / " + order.job().documentName();
    }

    @Override
    protected List<DocumentField> fields(PrintOrder order) {
        List<DocumentField> fields = new ArrayList<>();
        for (LineItem item : order.items()) {
            fields.add(new DocumentField(item.name(), LineItem.formatTenge(item.amount())));
        }
        fields.add(new DocumentField("Total", LineItem.formatTenge(order.totalCost())));
        return fields;
    }
}
```

campusprint/domain/PrintJob.java

```
package campusprint.domain;

public final class PrintJob {
    private final String documentName;
    private final int totalPages;
    private final int fromPage;
    private final int toPage;
    private final int copies;

    private PrintJob(Builder builder) {
        documentName = builder.documentName;
        totalPages = builder.totalPages;
        fromPage = builder.fromPage;
        toPage = builder.toPage;
        copies = builder.copies;
    }
```

```
    public String documentName() {
        return documentName;
    }

    public int totalPages() {
        return totalPages;
    }

    public int fromPage() {
        return fromPage;
    }

    public int toPage() {
        return toPage;
    }

    public int copies() {
        return copies;
    }

    public int pagesPerCopy() {
        return toPage - fromPage + 1;
    }

    public int printedPages() {
        return pagesPerCopy() * copies;
    }

    public static final class Builder {
        private final String documentName;
        private final int totalPages;
        private int fromPage = 1;
        private int toPage;
        private int copies = 1;

        public Builder(String documentName, int totalPages) {
            this.documentName = documentName;
            this.totalPages = totalPages;
            this.toPage = totalPages;
        }

        public Builder range(int from, int to) {
            fromPage = from;
            toPage = to;
            return this;
        }

        public Builder copies(int count) {
            copies = count;
            return this;
        }

        public PrintJob build() {
            if (documentName == null || documentName.isBlank()) {
                throw new IllegalArgumentException("Enter a document name.");
            }
            if (totalPages < 1 || copies < 1) {
                throw new IllegalArgumentException("Page and copy counts must be positive.");
            }
            if (fromPage < 1 || toPage < fromPage || toPage > totalPages) {
                throw new IllegalArgumentException("Select a range within pages 1-" + totalPages
+ ".");
            }
            if ((long) (toPage - fromPage + 1) * copies > Integer.MAX_VALUE) {
                throw new IllegalArgumentException("Print volume exceeds the per-job page
```

```
limit.");
            }
            return new PrintJob(this);
        }
    }
}
```

campusprint/domain/PrintOrder.java

```
package campusprint.domain;

import campusprint.pricing.LineItem;
import campusprint.pricing.PrintService;
import java.util.List;
import java.util.Objects;

public final class PrintOrder {
    private final int id;
    private final PrintJob job;
    private final PrintService service;

    public PrintOrder(int id, PrintJob job, PrintService service) {
        if (id <= 0) {
            throw new IllegalArgumentException("Order IDs must be positive.");
        }
        this.id = id;
        this.job = Objects.requireNonNull(job, "job");
        this.service = Objects.requireNonNull(service, "service");
    }

    public int id() {
        return id;
    }

    public PrintJob job() {
        return job;
    }

    public PrintService service() {
        return service;
    }

    public int printedPages() {
        return job.printedPages();
    }

    public String summary() {
        int pages = printedPages();
        return "#%d %s (%d %s)".formatted(id, job.documentName(), pages, pages == 1 ? "page" :
"pages");
    }

    public List<LineItem> items() {
        return List.copyOf(service.itemsFor(job));
    }

    public long totalCost() {
        return items().stream().mapToLong(LineItem::amount).sum();
    }

    @Override
    public String toString() {
        return "#%d %s | copies: %d | pages: %d | %s".formatted(
                id, job.documentName(), job.copies(), printedPages(),
```

```
LineItem.formatTenge(totalCost()));
    }
}
```

campusprint/observer/PrintJournal.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;
import java.util.ArrayList;
import java.util.List;

public final class PrintJournal implements PrintSubscriber {
    private final List<String> entries = new ArrayList<>();

    @Override
    public void onPrinted(PrintOrder order) {
        entries.add("#" + order.id() + " " + order.job().documentName());
    }

    public List<String> entries() {
        return List.copyOf(entries);
    }

    public String summary() {
        return "Journal: " + (entries.isEmpty() ? "(empty)" : String.join(", ", entries));
    }
}
```

campusprint/observer/PrintStatistics.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;

public final class PrintStatistics implements PrintSubscriber {
    private int completedOrders;
    private long totalPrintedPages;

    @Override
    public void onPrinted(PrintOrder order) {
        completedOrders++;
        totalPrintedPages += order.printedPages();
    }

    public int completedOrders() {
        return completedOrders;
    }

    public long totalPrintedPages() {
        return totalPrintedPages;
    }

    public String summary() {
        return "Completed orders: %d | Printed pages: %d".formatted(completedOrders,
totalPrintedPages);
    }
}
```

campusprint/observer/PrintSubscriber.java

```
package campusprint.observer;

import campusprint.domain.PrintOrder;

@FunctionalInterface
```

```
public interface PrintSubscriber {
    void onPrinted(PrintOrder order);
}
```

campusprint/pricing/BasicPrinting.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.List;

public final class BasicPrinting implements PrintService {
    public static final int PRICE_PER_PAGE = 20;
    public static final String SERVICE_NAME = "Printing";

    @Override
    public List<LineItem> itemsFor(PrintJob job) {
        return List.of(new LineItem(SERVICE_NAME, (long) job.printedPages() * PRICE_PER_PAGE));
    }
}
```

campusprint/pricing/Binding.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;

public final class Binding extends PrintServiceDecorator {
    public static final int PRICE_PER_COPY = 200;
    public static final String SERVICE_NAME = "Binding";

    public Binding(PrintService wrapped) {
        super(wrapped, SERVICE_NAME);
    }

    @Override
    protected long costFor(PrintJob job) {
        return (long) job.copies() * PRICE_PER_COPY;
    }
}
```

campusprint/pricing/Cover.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;

public final class Cover extends PrintServiceDecorator {
    public static final int PRICE_PER_COPY = 100;
    public static final String SERVICE_NAME = "Cover";

    public Cover(PrintService wrapped) {
        super(wrapped, SERVICE_NAME);
    }

    @Override
    protected long costFor(PrintJob job) {
        return (long) job.copies() * PRICE_PER_COPY;
    }
}
```

campusprint/pricing/LineItem.java

```
package campusprint.pricing;

import java.util.Locale;
```

```
public record LineItem(String name, long amount) {
    public LineItem {
        if (name == null || name.isBlank() || amount < 0) {
            throw new IllegalArgumentException("Each charge needs a name and a nonnegative
amount.");
        }
    }

    public static String formatTenge(long amount) {
        return String.format(Locale.ROOT, "%,d KZT", amount);
    }
}
```

campusprint/pricing/PrintService.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.List;

public interface PrintService {
    List<LineItem> itemsFor(PrintJob job);
}
```

campusprint/pricing/PrintServiceDecorator.java

```
package campusprint.pricing;

import campusprint.domain.PrintJob;
import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public abstract class PrintServiceDecorator implements PrintService {
    private final PrintService wrapped;
    private final String name;

    protected PrintServiceDecorator(PrintService wrapped, String name) {
        this.wrapped = Objects.requireNonNull(wrapped, "wrapped");
        this.name = name;
        PrintService current = wrapped;
        while (current instanceof PrintServiceDecorator finish) {
            if (finish.name.equals(name)) {
                throw new IllegalArgumentException(name + " is already included.");
            }
            current = finish.wrapped;
        }
    }

    protected abstract long costFor(PrintJob job);

    @Override
    public final List<LineItem> itemsFor(PrintJob job) {
        List<LineItem> items = new ArrayList<>(wrapped.itemsFor(job));
        items.add(new LineItem(name, costFor(job)));
        return List.copyOf(items);
    }
}
```

campusprint/queue/FirstComeFirstServed.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;
```

```
public final class FirstComeFirstServed implements QueuePolicy {
    @Override
    public Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders) {
        if (pendingOrders == null || pendingOrders.isEmpty()) {
            return Optional.empty();
        }
        return Optional.of(pendingOrders.get(0));
    }

    @Override
    public String name() {
        return "First come, first served";
    }
}
```

campusprint/queue/QueuePolicy.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;

public interface QueuePolicy {
    /** The input is in arrival order. Return empty only if the queue is empty. */
    Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders);

    String name();
}
```

campusprint/queue/ShortestJobFirst.java

```
package campusprint.queue;

import campusprint.domain.PrintOrder;
import java.util.List;
import java.util.Optional;

public final class ShortestJobFirst implements QueuePolicy {
    @Override
    public Optional<PrintOrder> selectNext(List<PrintOrder> pendingOrders) {
        if (pendingOrders == null || pendingOrders.isEmpty()) {
            return Optional.empty();
        }
        PrintOrder shortest = pendingOrders.get(0);
        for (PrintOrder order : pendingOrders) {
            if (order.printedPages() < shortest.printedPages()) {
                shortest = order;
            }
        }
        return Optional.of(shortest);
    }

    @Override
    public String name() {
        return "Shortest job first";
    }
}
```